

EE 446: Tiny Machine Learning

Lab 5 Keyword Spotting Using TinyML

Name: Kourosh | Student ID: 2331288

Public Edge Impulse Project: <https://studio.edgeimpulse.com/public/1060255/live>

1. Why convert raw audio to spectrogram/MFCC features?

Raw audio is a long, high-dimensional time series a one-second clip at 16 kHz is 16,000 samples and in the time domain the features that distinguish one spoken word from another are not directly exposed, so “yes” and “no” look like nearly identical waveforms. Converting to a spectrogram/MFCC representation extracts the frequency content that actually characterizes speech, turning each word into a distinct, separable time–frequency pattern. It also compresses the signal dramatically: the notebook’s `micro` preprocessing produces a compact $40 \times 49 = 1,960$ -value feature map small enough to fit in the Arduino’s limited memory. Finally, the Mel scaling warps the frequency axis to match human hearing, emphasizing the frequencies that carry speech information. The result is a small, decorrelated “image” that a lightweight CNN can learn from far more efficiently and robustly than it could from the raw waveform.

2. Purpose of the representative dataset in INT8 quantization

Full integer quantization maps 32-bit floating-point values onto just 256 integer levels, which requires knowing the numeric range of every tensor in the network. The weights are static, so their ranges are known directly, but the activations depend on the input data and cannot be known without running data through the model. The representative dataset provides that data: the converter feeds these realistic samples through the graph, observes the actual range of activation values at each layer, and uses those ranges to compute the scale and zero-point that map float to int8. In the notebook, `representative_dataset_gen()` yields 100 real feature vectors from the test set for exactly this calibration. Without it, the converter cannot produce a fully integer model and good calibration is what keeps the INT8 model close to the float accuracy here **92.23% quantized vs. 92.07% float**, with the model shrinking from 68,380 to 18,984 bytes.

3. What does `audio_processor.get_data()` do?

`get_data()` is the method on the `AudioProcessor` class in `input_data.py` that assembles a batch of preprocessed feature vectors and their labels for a chosen data split. Given the number of samples to return (with `-1` meaning all), an offset, the model settings, the background-noise and time-shift augmentation parameters, and the mode (`training/validation/testing`), it selects the requested clips, applies augmentation by mixing in background noise and randomly shifting each clip in time, and runs every clip through the same `micro`/MFCC feature-extraction pipeline the model expects. It returns two arrays: the feature data (shape $N \times 1,960$) and the integer labels. This is why the notebook calls it in three places to build the representative set for quantization and to pull the full test set for both the float and quantized accuracy evaluations.

4. What are `g_model` and `g_model_len`?

`g_model` is a C/C++ array (`alignas(8) const unsigned char g_model[]`) that holds the entire quantized `.tflite` model as its raw bytes, written one hex value at a time by `tflite_to_c_array()`, and `g_model_len` is the integer giving its exact size in bytes (18,984 here). The Arduino has no filesystem to load a `.tflite` file from, so TensorFlow Lite for Microcontrollers reads the model directly out of this byte array in flash: `GetModel(g_model)` parses the bytes and the length tells the runtime how many bytes to read. Both must be copied verbatim into `micro_features_model.cpp`, because a truncated array is a corrupt model that fails to parse and a mismatched length makes the interpreter read the wrong number of bytes and crash. This is exactly why the model must be copied from the generated `kws.cc` file rather than the truncated cell preview, which shows only a handful of the 18,984 bytes.

5. Serial Monitor keyword identification

The screenshot below shows the Arduino IDE Serial Monitor correctly identifying spoken keywords after deploying the quantized KWS model on the Arduino Nano 33 BLE Sense.

```
16:32:10.846 -> Heard unknown (205) @155200ms
16:32:12.927 -> Heard yes (204) @157264ms
16:32:15.859 -> Heard unknown (208) @160224ms
16:32:18.556 -> Heard unknown (204) @162912ms
16:32:23.301 -> Heard unknown (209) @167664ms
16:32:27.811 -> Heard unknown (205) @172160ms
16:32:29.327 -> Heard unknown (241) @173680ms
16:32:30.846 -> Heard unknown (225) @175200ms
16:32:31.734 -> Heard no (212) @176096ms
16:32:33.712 -> Heard unknown (210) @178080ms
```